

Version Controlling Systems and Git

Thejas Padubidri Rao
School of Electrical Engineering
MIT Manipal
Manipal, Karnataka
xxxxxxxxx@gmail.com

Abstract— Git is a distributed version control system essential for tracking code changes and facilitating collaboration. While introductory documentation often reduces Git to a rudimentary sequence of commands, recovering from complex errors requires a deep understanding of its internal mechanics. This extended report provides an inside-out study of Git, detailing its architectural reliance on blobs, trees, and commits. Structured as an advanced, offline survival guide, it translates theoretical mechanics into actionable workflows covering branching, interactive rebasing, micro-corrections, offline diagnostics, and critical emergency recovery operations.

Keywords—Git, GitHub, DVCS, Repositories

I. INTRODUCTION & CORE MECHANICS

Unlike centralized systems, Git is a Distributed Version Control System (DVCS), meaning every cloned repository is a complete, self-contained backup containing the entire project history. To utilize Git effectively, especially when disconnected from a network, a developer must understand the three primary local states that files can reside in:

- [1] Working Directory: The local file system where you edit files.
- [2] Staging Area (Index): A pre-commit holding area. Files here are marked to be included in the next snapshot.
- [3] Repository (HEAD): The database where Git permanently stores your committed snapshots.

II. ABOUT VERSION CONTROLLING SYSTEMS

A version control system, or VCS, tracks the history of changes as people and teams collaborate on projects together. As developers make changes to the project, any earlier version of the project can be recovered at any time. VCSs give each contributor a unified and consistent view of a project, surfacing work that's already in progress. Seeing a transparent history of changes, who made them, and how they contribute to the development of a project helps team members stay aligned while working independently.

A. Basic VCS Workflow:-

Modify → Stage → Commit → Push

1) Commands

git status

git add <file>

git commit -m "message"

git push

III. INTERNAL ARCHITECTURE MODEL

To recover from catastrophic mistakes, one must understand that Git does not store "diffs" (changes between files). Instead, Git is a content-addressable file system. It stores complete snapshots using three fundamental data objects, all identified by a 40-character SHA-1 hash.

- Blobs (Binary Large Objects): Represents file contents. Git hashes the contents of a file (not the filename) and stores it as a blob.
- Trees: Represents directories. A tree object points to multiple blobs (files) and other trees (subdirectories), storing their filenames and permissions.
- Commits: A commit object points to the root tree of your project at that moment in time. It also contains the author, timestamp, commit message, and a pointer to its parent commit(s).

A. Anatomy of the .git Directory

All of Git's magic resides in the hidden .git folder at the root of your project. If this folder is deleted, your project's history is permanently lost. Understanding its key subdirectories allows for advanced manual recovery:

- objects/: The cryptographic database storing all blobs, trees, and commits.
- refs/: Contains pointers to commits (branches in refs/heads/ and tags in refs/tags/).
- HEAD: A text file indicating the currently active branch (e.g., ref: refs/heads/main).
- config: Project-specific configuration settings, including remote repository URLs.

IV. DISTRIBUTED VERSION CONTROLLING SYSTEMS AND GIT

In a distributed version control system, every developer has a full copy of the project and project history. Unlike once popular centralized version control systems, DVCSs don't need a constant connection to a central repository. Git is the most popular distributed version control system. Git is commonly used for both open source and commercial software development, with significant benefits for individuals, teams and businesses.

A. Advantages of using Git

- Git lets developers see the entire timeline of their changes, decisions, and progression of any project in one place. From the moment they access the history of a project, the developer has all the context they need to understand it and start contributing.

- Developers work in every time zone. With a DVCS like Git, collaboration can happen any time while maintaining source code integrity. Using branches, developers can safely propose changes to production code.
- Businesses using Git can break down communication barriers between teams and keep them focused on doing their best work. Plus, Git makes it possible to align experts across a business to collaborate on major projects.

B. About Repositories

A repository, or Git project, encompasses the entire collection of files and folders associated with a project, along with each file's revision history. The file history appears as snapshots in time called commits. The commits can be organized into multiple lines of development called branches. Because Git is a DVCS, repositories are self-contained units and anyone who has a copy of the repository can access the entire codebase and its history. Using the command line or other ease-of-use interfaces, a Git repository also allows for: interaction with the history, cloning the repository, creating branches, committing, merging, comparing changes across versions of code, and more.

1) Creating and Cloning Repositories

```
git init
git clone <repository-url>
```

V. GIT ARCHITECTURE AND INTERNAL DATA MODEL

To understand Git beyond its basic commands, it is important to understand how Git represents and stores information internally. A Git repository is not simply a folder containing different copies of a project. Instead, Git maintains an object database containing the information required to reconstruct different versions of a project.

The three primary objects used by Git are blobs, trees, and commits. These objects are identified using cryptographic hash values. Git also uses references, such as branches and tags, to provide convenient names for particular commits.

A. Commits and Storage

A commit is essentially a snapshot of the repository's state at a given time. Internally, Git stores file contents as "blobs" (Binary Large Objects) and directory structures as "trees". A commit object contains metadata (author, timestamp, message) and points to the root tree of the snapshot, as well as its parent commit(s).

1) Basic Repository Commands

```
git status
git add <file>
git commit -m "commit message"
```

directory → staging area → repository

VI. BRANCHING, MERGING AND HISTORY

A. Branching and Merging

A branch is simply a movable pointer to a commit. Merging combines multiple sequences of commits into a unified history. When changes conflict (e.g., modifying the same lines in a file), a "merge conflict" occurs. Git pauses the merge process, requiring the

developer to manually resolve the conflict and commit the finalized file.

1) Branch Commands

```
git branch
git branch <branch-name>
git switch <branch-name>
git switch -c <branch-name>
git branch -d <branch-name>
```

B. Rebasing vs. Merging

While merging preserves the exact history of branching, rebasing rewrites history by applying commits from a feature branch directly onto the tip of another branch (like main). This creates a clean, linear project history, though it alters commit hashes and should be avoided on public/shared branches.

1) Commands

```
git rebase <branch-name>
git rebase --continue
git rebase --abort
git switch main
git merge <branch-name>
git merge --abort
```

C. Stashing

When working on a feature, a developer might need to abruptly switch branches without committing half-finished work. The git stash command temporarily shelves uncommitted changes, leaving a clean working directory without losing progress.

1) Commands

```
git stash
git stash list
git stash pop
git stash apply
git stash drop
```

D. Logs and Reflogs

The git log command allows developers to traverse and view the commit history. However, Git also maintains a "reference log" (git reflog) that records where branch tips and the HEAD pointer were recently located over time locally. Reflogs serve as an safety net for recovering lost commits.

1) Commands

```
git log
git log --oneline
git log --oneline --graph --all
git reflog
```

2) Recovery

```
git reflog
git reset --hard <commit>
```

E. Resetting and Reverting

```
git reset <commit>
git reset --soft <commit>
git reset --mixed <commit>
git reset --hard <commit>
```

```
git revert <commit>
```

1) Process Diagram

$A \rightarrow B \rightarrow C$

revert C

$A \rightarrow B \rightarrow C \rightarrow C'$

VII. REMOTE OPERATIONS

Git allows repositories to sync via remote connections. "Pushing" uploads local commits to a remote server, updating its references. Conversely, "pulling" fetches data from the remote and automatically merges it into the local branch. Occasionally, a developer may need to overwrite remote history using a "force push". This is a dangerous operation that overwrites the remote reference regardless of its current state and requires caution.

A. Remote Repositories

```
git remote
git remote -v
git remote add origin <url>
git remote remove origin
```

B. Pushing

```
git push
git push origin main
git push -u origin main
```

C. Fetching

```
git fetch
git fetch origin
git fetch --all
```

ACKNOWLEDGMENTS

I would like to express my gratitude to FORMULA MANIPAL, for the assignment that prompted this deep dive into Git's architecture. I also acknowledge the creators of the *learngitbranching.js.org* tutorial, which significantly aided in visualizing complex branching and merging operations.

REFERENCES

- [1] P. Cottle, "Learn Git Branching," learngitbranching.js.org.
- [2] GitHub, "About Git - GitHub Docs," [docs.github.com](https://docs.github.com/en/get-started/using-git/about-git).
- [3] C. Schafer, "Git Tutorial for Beginners: Command-Line Fundamentals," YouTube, Aug. 3, 2015. <https://www.youtube.com/watch?v=HVsySz-h9r4>